

CPF 배치 개발자 가이드

신규 업무 Job 작성·테스트·확장 시 바로 찾는 실무 매뉴얼

Core Platform Framework

기준: 최신 로컬 Source / 2026-08-16

전체 목차

장	찾을 수 있는 내용
1. 배치 개발 Quick Map	처리 모델·실행 방식·실패 후 조치를 빠르게 선택
2. 실행·테스트 명령	Batch Runtime, 업무 모듈 Test, 빠른/전체 검증
3. 처리 모델·Topology 선택	Tasklet·Chunk·Local/Remote 분산 방식 선택
4. Job·Step	@CpfBatchJob·@CpfBatchStep 기본 개발 계약
5. JobParameter	식별 Parameter·민감값·Validation 설계
6. Tasklet·Chunk·Reader/Processor/Writer	업무 처리 코드 배치와 Commit 단위
7. Transaction·Checkpoint	Local TX 범위와 Restart 시작점
8. 실행 요청 API	run/scheduledRun/retry/restart/rerun/stop/onDemand
9. Control Plane API	start/stop/restart/abandon/reconcile 공개 제어 계약
10. Retry·Restart·Rerun·Reprocess·Reconcile	실패 후 기능을 섞지 않고 선택
11. Partition·Worker·Lease·Fencing	대량 처리·Remote Worker 확장과 중복 차단
12. Scheduler·Misfire	예약 실행·중복 Trigger·누락 실행 처리
13. Center·Cut	대상 선정·Preview·분할·처리·최종 대사
14. 외부 연계·UNKNOWN RESULT	외부 Side Effect 와 결과불명 처리
15. Batch 실행 Property	Chunk/Partition/Remote/Executor 튜닝 값과 범위
16. Test·ADM·운영 인계	필수 Test 와 운영자가 알아야 할 정보
17. Batch 개발 기능 TOP 50	공개 API·옵션·실행 개념 Quick Reference

1. 배치 개발 Quick Map

[전체 목차로](#)

처리량, 재시작 필요 여부, Remote Worker 사용 여부와 실패 후 조치가 배치 설계의 핵심 선택입니다. 아래 Summary 에서 먼저 처리 모델과 후속 기능을 정한 뒤 해당 장에서 코드와 옵션을 확인합니다.

개발 목적	선택	핵심 판단	상세
단순 1 회 처리	Tasklet	한 Step 에서 명확히 완료되는 작업	§3-6
대량 Record 처리	Chunk	Reader/Processor/Writer 와 Commit 단위 필요	§3-6·7
같은 Runtime 병렬	LOCAL_PARTITION	Partition Key 로 안전하게 분할 가능	§3-11
Remote Worker 분산	REMOTE_PARTITION	Lease/Claim/Fencing 이 필요한 대량 처리	§3-11
Remote Chunk	REMOTE_CHUNK	Chunk 전송 중 Duplicate/Timeout/UNKNOWN 고려	§3-11
Remote Step	REMOTE_STEP	Step 전체를 별도 Runtime 에서 실행	§3-11
대량 대상 선정·대사	Center-Cut	Preview 부터 최종 대사까지 필요	§13

실패 후 기능은 의미가 다릅니다

기능	사용 목적	판단 기준	상세
Retry	같은 작은 처리의 일시 오류 재시도	역등하고 재시도 성공 가능성이 높음	§10
Restart	기존 JobInstance 를 Checkpoint 이후 재개	FAILED/STOPPED + restartable	§10
Rerun	새 JobInstance 로 전체 다시 실행	전체 재수행이 필요하고 중복 Side Effect 를 통제 가능	§10
Reprocess	실패/선택 업무 건만 재처리	대상 건을 정확히 식별 가능	§10
Reconcile	결과불명/불일치를 실제 상태로 확정	UNKNOWN_RESULT/Partial/응답 유실	§10-14
Abandon	해당 Execution 을 Restart 대상에서 제외	운영 정책상 더 이상 이어 실행하지 않음	§9-10

2. 실행·테스트 명령

[전체 목차로](#)

실행 위치: 프로젝트 Root | Script: `.\cpf-tools\build\tools\cpf-dev.ps1` | Gradle Wrapper: `.\gradlew.bat`

2.1 로컬 실행·검증

용도	명령 / Task	대상·주요 옵션	참고사항
Batch Runtime 만 실행	<code>cpfRunBatch</code>	Root Gradle Task	업무 Batch 개발 시 필요한 Runtime 만 기동
Batch Script 실행	<code>cpf-dev.ps1 run-batch</code>	<code>-ResourceProfile local/dev/test/stg/prod</code>	동일 Script 체계 사용
특정 업무 모듈 Test	<code>gradlew -p <module> test</code>	예: <code>cpf-member</code> / <code>cpf-external</code>	변경한 Job/업무 코드 빠른 확인
전체 Java Test	<code>cpf-dev.ps1 test</code>	<code>-ResourceProfile</code>	회귀 확인
빠른 검증	<code>cpf-dev.ps1 verify-fast</code>	<code>-ResourceProfile</code>	계약·Catalog·정적 Gate
전체 로컬 검증	<code>cpf-dev.ps1 verify-full</code>	<code>-ResourceProfile</code> / <code>-OutputRoot</code>	DB·Runtime 포함 최종 확인
상태 / 종료	<code>cpf-dev.ps1 status / stop</code>	Local Runtime	기동 여부 확인 / 개발 종료

```
# Batch Runtime 만 기동
.\gradlew.bat cpfRunBatch

# Batch 개발 Script
pwsh .\cpf-tools\build\tools\cpf-dev.ps1 run-batch -ResourceProfile local

# 변경한 업무 Domain 만 Test
.\gradlew.bat -p cpf-member test
```

2.2 Job 실행·운영 제어

용도	사용 경로	기준	상세
신규 실행	ADM Batch 실행 / <code>CpfBatchExecutionRequest.run()</code>	Job + Parameter	\$8
예약 실행	Scheduler / <code>scheduledRun()</code>	Schedule + Job + Parameter	\$8-12
정상 중지	ADM Execution / <code>stop()</code>	실행 중 상태	\$8-9
Restart	Recovery / <code>restart()</code>	실패 Execution + Checkpoint	\$8~10
Rerun	Recovery / <code>rerun()</code>	새 JobInstance 전체 실행	\$8-10
Reconcile	Recovery / <code>reconcile()</code>	UNKNOWN_RESULT	\$9-10-14

3. 처리 모델·Topology 선택

[전체 목차로](#)

배치 Job 개발 표준 흐름

처리 성격에 맞게 Tasklet/Chunk를 선택하고 Transaction·Checkpoint·Restart 가능성을 함께 설계합니다.



업무량 증가 시 Worker/Agent를 추가하고, 운영 제어와 실제 Job 처리를 분리합니다.
Restart와 Reprocess는 같은 동작이 아니므로 Job의 Side Effect와 Checkpoint를 기준으로 선택합니다.

Topology	처리 방식	적합한 경우	반드시 확인
LOCAL	단일 Runtime 실행	일반 Job/낮은 병렬 요구	Local TX / Metadata
PARALLEL_STEPS	독립 Step 병렬	Step 간 데이터 의존이 낮음	공유 데이터 충돌
LOCAL_PARTITION	동일 Runtime Partition 병렬	분할 가능한 대량 처리	Partition Key / Thread / Resource
REMOTE_PARTITION	별도 Worker Partition 분산	수평 확장이 필요한 대량 처리	Lease / Claim / Fencing / Takeover
REMOTE_CHUNK	Chunk 를 Remote Worker 에 분산	Chunk 병렬성과 중앙 Reader/Writer 패턴	Duplicate / Timeout / Retry / UNKNOWN
REMOTE_STEP	Step 전체 Remote 실행	특정 Step 을 별도 Runtime/Artifact 로 수행	Artifact/Worker Version / 실행 상태

4. Job·Step

전체 목차로

일반 업무 Job 은 `cpf-starter-batch`를 사용하고, Job/Step 의 안정적인 ID 와 Restart/동시 실행 정책을 Annotation 으로 선언합니다. Worker/Runtime 내부 API 를 업무 Job 에서 직접 조립하지 않습니다.

구분	사용 API / Annotation	용도	주요 옵션·기준
Job	@CpfBatchJob	CPF Job Catalog/Context/운영 추적 등록	value / restartable=true / maxConcurrentExecutions=1
Step	@CpfBatchStep	Job 내부 Step Handler 표시	value / order=0 / idempotent=true
확장 SPI	BatchStepHandler	FILE/SHELL/API Executor Adapter	일반 업무 Job 보다 Worker/Adapter 개발 시 사용

```
@CpfBatchJob(
    value = "MBR_MEMBER_DAILY",
    restartable = true,
    maxConcurrentExecutions = 1
)
public class MemberDailyJob {
    @CpfBatchStep(value = "LOAD_MEMBER", order = 10, idempotent = true)
    public void loadMember() { ... }
}
```

4.1 같은 이름의 @CpfBatchJob 구분

사용 패턴	Import	주요 속성	선택 기준
Annotation Job/Step 모델	com.cpf.batch.api.annotation.CpfBatchJob	value / restartable / maxConcurrentExecutions	신규 Annotation Step Handler 방식
Spring Batch Job Bean 메타데이터	com.cpf.batch.api.CpfBatchJob	id / name / ownerDomain	Job Bean 기반 Education/기존 구성

Import 확인 두 계약은 이름이 같지만 Target 과 속성이 다릅니다. 코드 예제를 적용할 때 import package 와 사용 패턴을 함께 확인합니다.

5. JobParameter

[전체 목차로](#)

JobParameter 는 단순 입력값이 아니라 JobInstance 식별, Restart/Rerun 의미, 민감정보 Masking 과 운영 화면에 영향을 줍니다.

항목	용도	선택 기준	주의
name	Parameter Key	영문 시작, 최대 100 자 Token	운영 중 의미를 바꾸지 않음
type	자료형/참조형	STRING/INTEGER/LONG/DECIMAL/ BOOLEAN/DATE/DATETIME/ENUM/ JSON_OBJECT 등	Secret 은 SECRET_REFERENCE 사용
required	필수 여부	실행 전 검증이 필요한 입력	Job 내부 중복 검증 최소화
defaultValue	기본값	입력 생략을 허용할 때	운영자가 예상 가능한 값
identifying	JobInstance 식별 참여	businessDate/tenant 등 논리 실행을 구분하 는 값	Restart/Rerun 의미에 직접 영향
sensitive	민감값 표시	Secret/민감 Parameter	Logging/운영화면 Masking
allowedValues/ pattern/min/max	사전 Validation	허용값·형식·범위 제한	실행 전 오류를 차단

6. Tasklet·Chunk·Reader/Processor/Writer

[전체 목차로](#)

상황	선택	개발 기준	TX/Restart 영향
파일 이동·정리·단일 Command	Tasklet	한 작업 단위로 명확하게 끝남	Step 단위 재실행
DB 대량 조회→변환→저장	Chunk	Reader/Processor/Writer 로 책임 분리	Chunk Commit/Checkpoint
외부 호출 대량 반복	Chunk + 멍등성	Side Effect 와 UNKNOWN 을 별도 설계	재시도 전 결과 판정
초대량 분할 처리	Partition + Chunk	안정적인 Partition Key	Partition 별 진행/재처리

Reader / Processor / Writer

역할	담당	피할 것	Restart 관점
Reader	안정된 정렬 Key 로 읽기	업무 Side Effect	Paging/Cursor 와 진행 위치
Processor	검증·변환·업무 계산	DB Commit 직접 제어	가능하면 순수 처리
Writer	DB Batch/Upsert/외부 반영	중복 방지 없는 재호출	멍등성과 Commit 단위

7. Transaction·Checkpoint

[전체 목차로](#)

Chunk 내부 DB 작업은 Local Transaction 으로 Commit 되지만, 외부기관 처리는 같은 DB Transaction 이 아닙니다. Chunk size 는 성능뿐 아니라 재실행 손실 범위와 Side Effect 비용을 함께 보고 정합니다.

설계 항목	기본 동작	실패 시	선택 기준
Chunk	Read → Process → Write → COMMIT	현재 Chunk rollback	처리량/Lock/재처리 비용
Checkpoint	완료된 진행 위치 저장	마지막 완료 지점부터 Restart	안정적인 Key/Cursor 만 저장
외부 호출	외부 Transaction 별도	응답 유실 시 UNKNOWN 가능	Idempotency + 결과 조회 + Reconcile
Commit 된 이전 Chunk	이미 완료	Restart 시 보존	성공 Side Effect 를 다시 만들지 않음

Transaction 경계 DB Rollback 은 외부기관의 이미 완료된 처리를 되돌리지 않습니다. 외부 Side Effect 가 있다면 Timeout, UNKNOWN_RESULT, Idempotency, Reconcile 을 함께 설계합니다.

8. 실행 요청 API

[전체 목차로](#)

업무/ADM/Scheduler 호출자는 Spring Batch JobLauncher 를 운영 호출 Surface 로 직접 노출하지 않고 `CpfBatchExecutionRequest`를 통해 실행 의도를 명확히 구분합니다.

요청	operationType	용도	주요 조건
run(...)	RUN	일반 신규 실행	lockRequired=true
scheduledRun(...)	SCHEDULE_RUN	Scheduler 예약 실행	scheduleId + jobId
retry(...)	RETRY	기존 실행 기준 일시 실패 재시도	sourceExecutionId
restart(...)	RESTART	Checkpoint 이후 기존 Instance 재개	FAILED/STOPPED + restartable
rerun(...)	RERUN	새 Instance 로 전체 재실행	중복 Side Effect 확인
stop(...)	STOP	정상 중지 요청	lockRequired=false; Kill 과 구분
onDemand(...)	RUN	Online 접수형 실행	standardBatchId/businessDate/ idempotencyKey

9. Control Plane API

[전체 목차로](#)

아래 계약은 ADM·Scheduler·Center·Cut 이 사용하는 공개 실행 생명주기 Surface 입니다. 일반 업무 Job 은 이 제어 로직을 직접 구현하지 않고 Runtime 에 위임합니다.

API	용도	입력/조건	반환
start(BatchApprovedLaunchRequest)	승인된 실제 실행 시작	승인/Launch 계약	BatchExecutionLink
stop(jobExecutionId, operatorId, reason)	정상 중지	실행 ID + 운영자 + 사유	boolean
restart(..., fencingToken)	Checkpoint 기반 재개	실행 ID + fencingToken	BatchExecutionLink
abandon(jobExecutionId, operatorId, reason)	Restart 대상에서 제외	운영 판단/사유	void
reconcile(cpfExecutionId)	Control State 와 실제 결과 대사	UNKNOWN_RESULT 대표 대상	BatchExecutionLink

Control State

상태	의미	다음 판단
RESERVED / STARTING / STARTED	실행 예약·기동·실행 중	중복 실행/heartbeat 확인
STOPPING / STOPPED	정상 중지 흐름	Restart 가능 여부 확인
COMPLETED	실행 성공 확정	건수/업무 대사 확인
FAILED	실패 확정	Retry/Restart/Reprocess 선택
UNKNOWN_RESULT	결과 미확정	Reconcile 먼저 수행
ABANDONING / ABANDONED	Restart 대상에서 제외 중/완료	새 실행 필요 여부 판단
REJECTED	실행 거절	권한/동시 실행/검증 원인 확인

10. Retry·Restart·Rerun·Reprocess·Reconcile

[전체 목차로](#)

기능	무엇을 다시 하나	언제 선택	주의
Retry	같은 작은 처리	일시 오류 + 멍등	UNKNOWN Blind Retry 금지
Restart	기존 JobInstance 의 미완료 구간	Checkpoint 가 신뢰 가능	앞서 Commit 된 Chunk 보존
Rerun	전체 Job 을 새 JobInstance 로	전체 재수행이 필요	이미 성공한 Side Effect 확인
Reprocess	실패/선택 업무 건	Business Key 로 대상 식별 가능	성공한 건 제외
Reconcile	실제 결과 확인/확정	UNKNOWN/Partial/Kill/응답 유실	Retry 가 아님
Abandon	Execution 을 Restart 대상에서 제외	더 이상 이어 실행하지 않을 때	새 실행과 구분

용어 기준 Restart, Rerun, Reprocess, Reconcile 은 서로 다른 기능입니다. 운영/교육/코드에서 같은 의미로 섞어 쓰지 않습니다.

11. Partition·Worker·Lease·Fencing

[전체 목차로](#)

Batch Runtime - 업무 증가에 따라 확장

실행 제어와 실제 Job 실행을 분리하고, 업무량이 늘면 Worker / Agent를 추가해 확장합니다.



업무량이 늘면 Worker/Agent 를 추가해 실행 용량을 확장할 수 있습니다. Remote 실행에서 가장 중요한 것은 같은 Partition 을 두 Worker 가 동시에 처리하지 않도록 Claim/Lease/Fencing 을 지키는 것입니다.

개념	역할	개발자가 정할 것	실패 시
Partition Key	처리 범위 분리	겹치지 않는 안정적인 Key	중복/누락 검증
Claim	한 Worker 가 Partition 소유	업무 코드에서 별도 소유권 구현하지 않음	소유권 충돌 차단
Lease	Worker 생존/소유 기간	Timeout 과 처리시간 관계 확인	만료 후 Takeover
Fencing Token	stale Worker 쓰기 차단	Side Effect 가 Token 을 존중하는지	오래된 Worker write 거절
Agent / Worker	실행 Node 확장	업무 Job 코드와 Runtime 제어 분리	Drain/재할당/재기동

12. Scheduler·Misfire

[전체 목차로](#)

항목	용도	선택 기준	운영 확인
Schedule	정기 실행 시각	Cron/주기 + Timezone + businessDate	다음 실행 시각
maxConcurrentExecutions	동일 Job 중복 실행 제한	업무상 병렬 허용 여부	중복 Trigger/수동 실행
Misfire	놓친 Trigger 처리	즉시 보정/건너뛸 등 정책	Trigger History
Enable/Disable	Schedule 활성화 상태	변경 권한/사유	Audit
Run Once	예약 외 1 회 실행	Parameter/중복 실행 확인	Execution/Audit

13. Center-Cut

[전체 목차로](#)

Center-Cut 은 대량 대상 선정부터 Preview, 분할, 실행, 실패 대상, 최종 대사를 하나의 흐름으로 관리할 때 사용합니다. `Job COMPLETED`와 `업무 대사 완료`는 같은 의미가 아닙니다.

단계	개발 기준	확인 값
Target 선정	실제 실행과 같은 대상 조건	원천 대상 건수
Preview	실행 전 대상/규모 확인	예상 건수/금액
Partition / Chunk	안정적인 분할 Key	중복/누락 없는 범위
Execute	Worker 처리	Success/Failure/Unknown
Reprocess	실패/선택 대상만	원 실행과 Business Key 연결
Reconcile	원천-처리 결과 최종 대사	Success + Failure + Excluded + Confirmed Unknown

14. 외부 연계·UNKNOWN_RESULT

[전체 목차로](#)

Batch 에서 외부 Side Effect 가 포함되면 DB Commit 과 외부 처리 결과가 분리됩니다. 응답을 받지 못한 경우 `FAILED`로 단정하거나 동일 요청을 바로 다시 보내지 않습니다.

상태	의미	개발 처리	다음
SUCCESS	성공 확정	완료 기록	건수/업무 대사
BUSINESS_FAILURE	업무 실패 확정	업무 사유 저장	재처리 가능 조건 판단
TECHNICAL_FAILURE	기술 실패 확정	Retry 정책 판단	먹등성/외부 상태 확인
UNKNOWN_RESULT	처리 여부 미확정	Recovery/Reconcile 연결	외부 조회/실제 DB 결과로 확정

15. Batch 실행 Property

[전체 목차로](#)

아래 값은 `cpf.batch.execution` 설정의 Source 기본값과 허용 범위입니다. 먼저 기본값으로 기능을 확인하고, 처리량/Worker/외부 SLA 근거가 있을 때만 변경합니다.

Property	기본값	허용 범위	용도
default-chunk-size	100	1..10,000	기본 Chunk size
max-partition-count	256	1..10,000	허용 최대 Partition 수
remote-poll-interval-ms	1,000	10..60,000	Remote 결과 Poll 간격
remote-timeout-ms	3,600,000	1,000..86,400,000	Remote 실행 Timeout
remote-chunk-max-wait-timeouts	40	1..10,000	Remote Chunk 최대 대기 Timeout 횟수
remote-chunk-throttle-limit	256	1..10,000	Remote Chunk 동시 처리 상한
max-materialized-jobs	1,024	16..100,000	Runtime 에 materialize 할 Job 상한
executor-core-size	4	1..512	Batch Executor core pool
executor-max-size	32	core..2,048	Batch Executor max pool
executor-queue-capacity	1,024	1..1,000,000	Batch Executor queue

튜닝 순서 Chunk/Partition 수부터 무작정 늘리지 말고 DB Lock/Connection, Worker 수, Remote Timeout, 외부 시스템 처리량을 함께 확인합니다.

16. Test·ADM·운영 인계

[전체 목차로](#)

Test	반드시 확인	Evidence
정상	전체 대상/건수/결과	Job/Step 상태
Parameter	required/type/range/sensitive	실행 전 Validation/Masking
Rollback	Chunk 중 Writer 실패	DB 결과/Checkpoint
Restart	중간 실패 후 이어서	중복·누락 없음
Rerun/Reprocess	전체 vs 선택 대상	성공 Side Effect 중복 없음
동시 실행	같은 Job/Parameter 중복 요청	maxConcurrent/Lease 제어
Worker Kill	Remote 처리 중 Process 종료	Lease/Takeover/Fencing
외부 UNKNOWN	응답 유실	Reconcile 후 결과 확정
Scheduler Misfire	실행 시각 누락	정책대로 보정

운영 인계 최소 정보

항목	인계 내용
Job	Job ID / 설명 / 담당 업무
Parameter	의미 / identifying / 민감값 / 예제
재실행	Retry/Restart/Rerun/Reprocess 가능 조건
외부 Side Effect	역등성 / UNKNOWN 조회 / Reconcile
처리량	예상 건수 / Chunk / Partition / Worker
Schedule	Timezone / businessDate / Misfire / 중복 실행
운영	ADM 확인 위치 / 권한 / 승인 / Audit

운영 호출 Surface 일반 업무 개발자는 Spring Batch `JobLauncher`를 운영 호출 API 로 직접 노출하지 않습니다. CPF 실행 요청/Control Plane 계약과 ADM/Scheduler 경로를 사용합니다.

17. Batch 개발 기능 TOP 50

전체 목차로

배치 개발자가 자주 판단하는 공개 계약과 실행 개념을 기능군별로 정리했습니다. 본문에서 먼저 처리 모델과 표준 흐름을 익히고, 이름이나 옵션을 빠르게 다시 찾을 때 사용합니다.

표기 Golden 은 일반 업무 개발자가 우선 익힐 항목, Capability 는 해당 기능을 사용할 때 확인할 항목, Advanced 는 Worker/Adapter/운영 제어 구현에서 주로 사용하는 계약입니다.

A. Job / Step 정의

No	구분	기능 / API	용도	주요 옵션·조건	상세
01	Golden	@CpfBatchJob	CPF Batch Job 등록	value / restartable=true / maxConcurrentExecutions=1	\$4
02	Golden	@CpfBatchJob.value()	안정적인 Job ID 지정	업무 Job ID 는 운영 중 임의 변경 금지	\$4
03	Golden	@CpfBatchJob.restartable()	Checkpoint 기반 Restart 허용 여부	기본 true; Checkpoint 설계와 함께 판단	\$4·10
04	Golden	@CpfBatchJob.maxConcurrentExecutions()	동일 Job 최대 동시 실행 수	기본 1; Scheduler/수동 Trigger 와 함께 판단	\$4·12
05	Golden	@CpfBatchStep	Job 내부 Step 선언	value / order / idempotent	\$4
06	Golden	@CpfBatchStep.value()	안정적인 Step ID 지정	Job 내부에서 식별 가능한 ID	\$4
07	Golden	@CpfBatchStep.order()	Step 순서 지정	기본 0	\$4
08	Golden	@CpfBatchStep.idempotent()	Retry/Remote 재전송 안전성 선언	기본 true; 실제 Side Effect 도 먹등해야 함	\$4·10

B. Step 실행 / Worker SPI

No	구분	기능 / API	용도	주요 옵션·조건	상세
09	Advanced	BatchStepHandler	FILE/SHELL/API 실행 Handler 확장	일반 업무 Job 보다 Worker/Adapter 개발용	\$11
10	Advanced	supports(executorType, reference)	Handler 지원 여부 판정	FILE / SHELL / API 등 Executor Type	\$11
11	Advanced	execute(BatchStepCommand)	실제 Step 실행	cpfExecutionId / fencingToken / parameters / context	\$11
12	Capability	COMPLETED	Step 성공 확정	BatchStepResult.Status	\$10·14
13	Capability	FAILED	Step 실패 확정	재시도 가능 여부와 별개	\$10·14
14	Capability	RETRYABLE_FAILURE	정책상 재시도 가능한 실패	모든 Exception 을 Retryable 로 만들지 않음	\$10
15	Capability	UNKNOWN_RESULT	실제 처리 결과 불명	Blind Retry 금지; Reconcile	\$10·14
16	Capability	STOPPED	정상 중지 흐름으로 종료	Process Kill 과 구분	\$10
17	Capability	readCount / writeCount / skipCount	처리량·대사 판단	ADM/운영 확인에 사용	\$16
18	Capability	checkpoint	Restart 가능한 진행 상태 저장	Map<String,Object>; 안정적인 위치만 저장	\$7

C. 실행 요청

No	구분	기능 / API	용도	주요 옵션·조건	상세
19	Golden	CpfBatchExecutionRequest.run()	일반 신규 실행 요청	RUN / lockRequired=true	\$8
20	Capability	scheduledRun()	Scheduler 예약 실행 생성	SCHEDULE_RUN / lockRequired=true	\$8·12
21	Capability	retry()	기존 실행 기준 일시 실패 재시도	sourceExecutionId; Restart/Rerun 과 구분	\$8·10
22	Golden	restart()	기존 JobInstance 를 Checkpoint 이후 재개	FAILED/STOPPED + restartable=true	\$8·10
23	Capability	rerun()	새 JobInstance 로 전체 재실행	성공 Side Effect 중복 여부 확인	\$8·10
24	Golden	stop()	정상 중지 요청	STOP / lockRequired=false; Kill 과 다름	\$8·10
25	Capability	onDemand()	Online 요청으로 Batch 접수	standardBatchId / businessDate / idempotencyKey	\$8
26	Capability	normalizedJobParameters()	Parameter JSON 정규화	blank -> {}	\$5·8
27	Capability	normalizedRequestUser()	요청자 정보 정규화	fallback 지원	\$8
28	Capability	normalizedReason()	실행/운영 사유 정규화	fallback 지원	\$8

D. Control Plane

No	구분	기능 / API	용도	주요 옵션·조건	상세
29	Advanced	BatchExecutionControlPort.start()	승인된 실제 Batch 실행 시작	BatchApprovedLaunchRequest -> BatchExecutionLink	\$9
30	Advanced	stop()	실행 중 정상 중지	jobExecutionId / operatorId / reason	\$9
31	Advanced	restart()	Checkpoint 기반 기존 실행 재개	fencingToken 포함	\$9
32	Advanced	abandon()	더 이상 Restart 대상이 아니게 종료	Cancel 과 의미가 다름	\$9·10
33	Advanced	reconcile()	Control State 와 실제 실행 결과 대사	UNKNOWN_RESULT 대표 대상	\$9·10

E. Job Parameter

No	구분	기능 / API	용도	주요 옵션·조건	상세
34	Golden	BatchParameterDefinition.name	Parameter Key	영문 시작 / 최대 100 자 Token	\$5
35	Golden	BatchParameterDefinition.type	Parameter 자료형	STRING/INTEGER/.../SECRET_REFERENCE 등	\$5
36	Golden	required	필수 Parameter 여부	Job 내부 중복 검증 최소화	\$5
37	Golden	defaultValue / effectiveValue()	입력 없을 때 기본값 결정	suppliedValue 우선	\$5
38	Golden	identifying	JobInstance 식별 참여 여부	Restart/Rerun 의미에 직접 영향	\$5
39	Golden	sensitive	민감 Parameter 표시	Logging/화면 Masking; Secret 원문 금지	\$5
40	Capability	allowedValues / pattern / min / max	Parameter 사전 Validation	길이·범위·정규식 포함	\$5

F. 실행 Topology

No	구분	기능 / API	용도	주요 옵션·조건	상세
41	Golden	LOCAL	단일 Runtime 에서 일반 Job 실행	Local TX / Metadata	\$3
42	Capability	PARALLEL_STEPS	동일 Runtime 에서 독립 Step 병렬	공유 데이터 충돌 확인	\$3
43	Capability	LOCAL_PARTITION	동일 Runtime Partition 병렬	Partition Key / Thread / Resource	\$3·11
44	Capability	REMOTE_PARTITION	별도 Worker 로 Partition 분산	Lease / Claim / Fencing / Takeover	\$3·11
45	Capability	REMOTE_CHUNK	Chunk 단위 Remote 분산	Duplicate / Timeout / Retry / UNKNOWN	\$3·11
46	Capability	REMOTE_STEP	Step 전체를 Remote Runtime 에서 실행	Artifact/Worker Version/Execution Status	\$3·11

G. 실패 후 처리

No	구분	기능 / API	용도	주요 옵션·조건	상세
47	Golden	Restart	기존 JobInstance 를 Checkpoint 부터 재개	앞서 Commit 된 Chunk 는 보존	\$10
48	Golden	Rerun	새 JobInstance 로 전체 다시 수행	중복 업무 반영/Idempotency 확인	\$10
49	Golden	Reprocess	실패/선택 업무 건만 별도 재처리	성공 건 제외; 원 실행과 Business Key 연결	\$10
50	Golden	Reconcile	상태/데이터의 실제 결과를 확인해 확정	UNKNOWN/Partial/Kill/응답 유실; Retry 가 아님	\$10·14